Erdős Problem #700(a) for integers with two prime factors, and exhaustive small-order checks of eight open Graffiti.pc conjectures

Contribution to the *Verifiable Mathematical Reasoning Corpus*
(prepared with AI assistance; all computations reproducible from the appendix)

November 2026

Abstract

For composite n let $f(n) = \min_{1<k\le n/2} \gcd\bigl(n, \binom{n}{k}\bigr)$ and let $P(n)$ be the largest prime factor of n. Erdős and Szekeres asked to characterize the composite n with $f(n) = n/P(n)$ (Erdős Problem #700(a)). This question is still open; on the problem's forum it has been described as probably admitting no satisfactory answer in general. We prove three things.

- A general necessary condition: if n is not a prime power, then $P(n)$ divides n exactly, and every other prime-power component $p^{v_p(n)}$ is smaller than $P(n)$.
- A complete characterization for every composite n with at most two distinct prime factors (Theorem 1.5). For $n = p^a q^b$ with $p < q$, equality holds if and only if $b = 1$, $p^a < q$, and an explicit base-p carry condition on small multiples of q is satisfied.
- An infinitude statement: for every prime power p^a, all primes $q > p^a$ with $q \equiv -1 \pmod{p^{2a}}$ satisfy $f(p^a q) = n/P(n)$.

The characterization agrees with brute force for all 11 237 such $n \le 30\,000$.

In a second, purely computational part we evaluate eight Graffiti.pc conjectures from *Written on the Wall II* that are open in the Formal Conjectures repository: 19, 40, 61, 72, 96, 133, 189 and 199. We use the repository's exact definitions and check every connected graph with at most ten vertices; no counterexample exists. We also verify, on the same graphs, the inequality $f(G) \ge \mathrm{res}(G) + \mathrm{im}(G)$, which was left open in corpus item [066] and implies WOWII 61. We record three short arguments: a proof of WOWII 96 (a proof with the same idea was posted earlier by another author), the reduction of WOWII 133 to C_4-free graphs, and WOWII 189 for graphs of diameter at most two.

None of these is a complete resolution of a named problem by the present work; the precise status of each item is stated where it appears and summarized in Section 4.

Contents

1 Erdős Problem #700(a) 2
- 1.1 Background . . . 2
- 1.2 Tools . . . 2
- 1.3 A necessary condition for general n . . . 2
- 1.4 The case of at most two prime factors . . . 3
- 1.5 Verification and data . . . 4

2 Eight open Graffiti.pc conjectures 4
- 2.1 Definitions . . . 4
- 2.2 Exhaustive verification up to ten vertices . . . 5
- 2.3 The residue–induced-matching inequality . . . 5
- 2.4 Three short arguments . . . 6

3 A status note on order-five Austin laws **6**

4 Assessment **7**

A Programs **7**

1 Erdős Problem #700(a)

1.1 Background

Erdős and Szekeres [1] introduced $f(n) = \min_{1<k\le n/2} \gcd(n, \binom{n}{k})$ for composite n and asked three questions, recorded as Erdős Problem #700 [2]:

(a) characterize the composite n with $f(n) = n/P(n)$;

(b) are there infinitely many composite n with $f(n) > n^{1/2}$?

(c) is $f(n) \ll_A n/(\log n)^A$ for every $A > 0$?

Question (b) has been answered affirmatively in 2026, by products of three primes; see the problem page and corpus item [086]. Questions (a) and (c) are listed as open (page last edited 28 August 2026). It is classical that $f(pq) = n/P(n)$ for $n = pq$, and that $f(30) = 6$. The forum discussion records further squarefree examples, such as 2310 and $71677177 = 11{\cdot}13{\cdot}19{\cdot}23{\cdot}31{\cdot}37$, and suggests that a general characterization is hopeless [2]. Our results give the complete answer for n with at most two prime factors.

1.2 Tools

For a prime ℓ and nonnegative integers x, y, let $\kappa_\ell(x, y)$ be the number of carries when x and y are added in base ℓ, and let $s_\ell(x)$ be the base-ℓ digit sum.

Lemma 1.1 (Kummer)**.** *$v_\ell\binom{x+y}{x} = \kappa_\ell(x, y) = \big(s_\ell(x) + s_\ell(y) - s_\ell(x + y)\big)/(\ell - 1)$. Moreover $\kappa_\ell(x, y) = 0$ if and only if every base-ℓ digit of x is at most the corresponding digit of $x + y$.*

Consequently

$$\gcd\Big(n, \binom{n}{k}\Big) = \prod_{\ell \mid n} \ell^{\min\{v_\ell(n),\, \kappa_\ell(k, n-k)\}}. \tag{1}$$

Lemma 1.2. *For $1 \le k \le n$ and every prime ℓ, $v_\ell\binom{n}{k} \ge v_\ell(n) - v_\ell(k)$.*

Proof. $k\binom{n}{k} = n\binom{n-1}{k-1}$. □

Lemma 1.3. *Let $\ell^e \parallel n$ with $e \ge 1$, and suppose $\ell^e \le n/2$. Then $\gcd(n, \binom{n}{\ell^e}) = n/\ell^e$.*

Proof. In base ℓ the lowest nonzero digit of n sits at position e. Adding ℓ^e to $n - \ell^e$ restores that digit without a carry, so $v_\ell\binom{n}{\ell^e} = 0$. For every other prime $r \mid n$, Lemma 1.2 gives $v_r\binom{n}{\ell^e} \ge v_r(n)$. □

1.3 A necessary condition for general n

Proposition 1.4. *Let n be composite and not a prime power, with $P = P(n)$. If $f(n) = n/P$, then $P \parallel n$, and $p^{v_p(n)} < P$ for every prime $p \mid n$ with $p \ne P$.*

Proof. Every prime-power component $\ell^e \parallel n$ satisfies $\ell^e \le n/2$, so Lemma 1.3 gives $f(n) \le n/\ell^e$. With $\ell = P$ this forces $e = 1$. With $\ell = p \ne P$ it gives $n/P \le n/p^{v_p(n)}$, i.e. $p^{v_p(n)} \le P$, and equality is impossible for distinct primes. □

For prime powers, $f(p^a) = p$ for all $a \ge 2$ (Theorem 1.5(i)). So among prime powers, $f(n) = n/P(n)$ holds exactly for the squares p^2.

1.4 The case of at most two prime factors

Theorem 1.5. *(i) If $n = p^a$ with $a \geq 2$, then $f(n) = p$. Hence $f(n) = n/P(n)$ if and only if $a = 2$.*

(ii) Let $n = p^a q^b$ with primes $p < q$ and $a, b \geq 1$, so that $P(n) = q$. Then $f(n) = n/q$ if and only if all of the following hold:

(1) $b = 1$;

(2) $p^a < q$;

(3) for every $t \in \{1, \ldots, a-1\}$ and every integer $j' \geq 1$ with $p \nmid j'$ and $p^t j' \leq p^a/2$,

$$\kappa_p(j'q,\ (p^{a-t} - j')q)\ \geq\ a.$$

Proof. (i) For $2 \leq k \leq p^a/2$ we have $v_p(k) \leq a-1$, so Lemma 1.2 gives $p \mid \gcd(p^a, \binom{p^a}{k})$. At $k = p^{a-1}$, adding p^{a-1} and $(p-1)p^{a-1}$ in base p produces exactly one carry, so the gcd equals p.

(ii) The values $k = q^b$ and $k = p^a$ are at most $n/2$, so Lemma 1.3 gives $f(n) \leq n/q^b$ and $f(n) \leq n/p^a$. If $b \geq 2$, then $f(n) < n/q$. If $p^a > q$, then $f(n) \leq n/p^a < n/q$. Hence (1) and (2) are necessary.

Assume (1) and (2). Then $f(n) \leq \gcd(n, \binom{n}{q}) = p^a$, so $f(n) = p^a$ if and only if $\gcd(n, \binom{n}{k}) \geq p^a$ for every $2 \leq k \leq n/2$. By (1) this gcd equals $p^{\min\{a, \kappa_p\}} q^{\min\{1, \kappa_q\}}$. If $\kappa_q(k, n-k) \geq 1$, the gcd is at least $q > p^a$.

Suppose $\kappa_q(k, n-k) = 0$. Since $p^a < q$, the base-q digits of $n = p^a q$ are $(p^a, 0)$. By Lemma 1.1, $k = jq$ with $0 \leq j \leq p^a$, and $2 \leq k \leq n/2$ means $1 \leq j \leq p^a/2$. For such k the gcd is $p^{\min\{a, \kappa_p(jq, (p^a - j)q)\}}$.

Write $j = p^t j'$ with $p \nmid j'$; then $t \leq a-1$. Multiplying both summands by p^t only shifts digits, so $\kappa_p(jq, (p^a - j)q) = \kappa_p(j'q, (p^{a-t} - j')q)$. If $t = 0$, Lemma 1.2 applied to $\binom{p^a q}{jq}$ gives at least a carries automatically. Hence the condition reduces exactly to (3). □

Corollary 1.6. *Let q be prime.*

1. *$f(pq) = n/P(n)$ for $p < q$ (classical: condition (3) is vacuous).*
2. *$f(4q) = 4 = n/P(n)$ for every prime $q \geq 5$.*
3. *$f(9q) = 9 = n/P(n)$ if and only if $q \geq 11$ and $s_3(2q) \geq 4$.*
4. *$f(8q) = 8 = n/P(n)$ if and only if $q \geq 11$, $s_2(q) \geq 3$ and $s_2(3q) \geq 3$.*
5. *For $n = p^2 q$ with $p^2 < q$: $f(n) = n/P(n)$ if and only if $\kappa_p(j'q, (p - j')q) \geq 2$ for all $1 \leq j' \leq p/2$.*

Proof. Specialize (3). For $p = 2, a = 2$ the only case is $t = 1$, $j' = 1$, with $\kappa_2(q, q) = s_2(q) \geq 2$. For $p = 3, a = 2$: $\kappa_3(q, 2q) = s_3(2q)/2$. For $p = 2, a = 3$ the cases are $(t, j') = (1, 1)$, giving $\kappa_2(q, 3q) = s_2(3q)$, and $(2, 1)$, giving $s_2(q)$. □

For example $f(104) = 8$ ($q = 13$), while $f(88) < 8$ ($s_2(33) = 2$) and $f(136) < 8$ ($s_2(17) = 2$).

Corollary 1.7 (Infinitude)**.** *Let p be prime and $a \geq 1$. Every prime $q > p^a$ with $q \equiv -1 \pmod{p^{2a}}$ satisfies $f(p^a q) = p^a = n/P(n)$. In particular, for every prime power p^a there are infinitely many primes q with $f(p^a q) = n/P(n)$; this includes integers with arbitrarily large non-squarefree part.*

Proof. Fix $t \in [1, a-1]$ and an admissible j', and put $x = j'q$, $y = (p^{a-t} - j')q$. A carry leaves position $i-1$ exactly when $(x \bmod p^i) + (y \bmod p^i) \geq p^i$. For $a - t \leq i \leq 2a$ we have $j' < p^{a-t} \leq p^i$ and $0 < p^{a-t} - j' < p^i$. Since $q \equiv -1 \pmod{p^i}$, this gives $x \bmod p^i = p^i - j'$ and $y \bmod p^i = p^i - p^{a-t} + j'$, with sum $2p^i - p^{a-t} \geq p^i$. That is $2a - (a-t) + 1 \geq a$ carries, so condition (3) holds. Conditions (1) and (2) hold by assumption, and Dirichlet's theorem supplies infinitely many such primes. □

1.5 Verification and data

A direct C program computes $f(n)$ from (1). Together with a Python implementation of Theorem 1.5 (Appendix A), it confirms the theorem for all 11 237 composite $n \leq 30\,000$ with at most two prime factors, with no mismatch. Outside the semiprimes pq, there are 7 722 composite $n \leq 30\,000$ with $f(n) = n/P(n)$.

For question (c), primorials are a natural test family. Here $f(n)$ falls rapidly below $n/P(n)$, consistent with a positive answer to (c):

n	$f(n)$	$n/P(n)$	$f(n)/(n/P(n))$
30	6	6	1
210	14	30	0.467
2310	154	210	0.733
30030	165	2310	0.0714
510510	510	30030	0.0170
9699690	2730	510510	0.0053

Status. Theorem 1.5 settles question (a) completely in the class $\omega(n) \leq 2$. Corpus item [086] settles certain products of three close primes. The general question (a) remains open.

2 Eight open Graffiti.pc conjectures

2.1 Definitions

We use the formalizations in the Formal Conjectures repository [3], where these eight statements are tagged "research open" as of 6 November 2026:

- $l(v)$: the independence number of $G[N(v)]$, with $\Lambda = \max_v l(v)$ and $\bar{l} = \frac{1}{n}\sum_v l(v)$;
- b: the largest order of an induced bipartite subgraph;
- f: the largest order of an induced forest;
- tree: the largest order of an induced tree;
- path: the largest order of an induced path;
- p: the path-cover number;
- res: the Havel–Hakimi residue;
- σ: the second-smallest degree;
- κ: the vertex connectivity;
- distEven(v): the number of vertices at even distance from v, counting v itself.

The conjectures checked are:

19: $b \geq \lfloor \bar{e} + \Lambda \rfloor$, where $\bar{e}$ is the average eccentricity;

40: $f \geq \lceil (p + b + 1)/2 \rceil$;

61: $f \geq \mathrm{res} + \lceil \mathrm{diam}\,/3 \rceil$;

72: $\mathrm{tree} \geq \lceil \bar{e} + \Lambda/3 \rceil$;

96: $\alpha \leq 1 + \min_v \mathrm{distEven}(v)\,(\Lambda - 1)$;

133: $\mathrm{path} \geq \mathrm{rad} + \bar{l}^{\,cC_4}$, where $cC_4 = 0$ if G contains a (not necessarily induced) 4-cycle and $cC_4 = 1$ otherwise;

189: if $\max_v \mathrm{distEven}(v) \leq 1 + \sigma$, then G has a Hamiltonian path;

199: if $\mathrm{tree} - 2 \leq \kappa$, then G has a Hamiltonian path.

2.2 Exhaustive verification up to ten vertices

Theorem 2.1 (computational)**.** *None of the eight conjectures above has a counterexample among the connected graphs with at most* 10 *vertices.*

The graphs were generated by `nauty-geng -c`; the counts of connected graphs for $n = 2, \ldots, 10$ are $1, 2, 6, 21, 112, 853, 11117, 261080, 11716571$. Every invariant is computed exactly by subset enumeration or dynamic programming (program `wow.c`, Appendix A). The program was cross-checked against independent `networkx` brute force on test graphs, including the Petersen graph.

Remark 2.2. In October 2026 several of these statements received public claims on the Formal Conjectures issue tracker:

- a counterexample to 72, with 31 vertices (issue #7662; an earlier record is reported there);
- proofs of 19 (pull requests #4559, #7571) and of 96 (issue #7664).

By Theorem 2.1, every counterexample to 72 has at least 11 vertices. We found no public claims for 61, 133, 189 or 199.

2.3 The residue–induced-matching inequality

Corpus item [066] reduced WOWII 61 to the following inequality and left it open:

$$f(G) \;\geq\; \mathrm{res}(G) + \mathrm{im}(G), \tag{2}$$

where im is the induced matching number. The reduction runs as follows. The edges with indices $0, 3, 6, \ldots$ of a diametral geodesic form an induced matching of size $\lceil \mathrm{diam}\,/3 \rceil$.

Proposition 2.3 (computational)**.** *Inequality* (2) *holds for every graph with at most* 10 *vertices.*

Proof. Connected graphs were checked exhaustively (program `12.c`). For disconnected graphs, f and im are additive over components and res is subadditive (corpus [066], Lemma "disjoint-union subadditivity"), so the connected case suffices. □

The variant with f replaced by α_2, the largest order of an induced subgraph of maximum degree at most one, already fails at 5 vertices.

2.4 Three short arguments

Theorem 2.4 (WOWII 96)**.** *For every connected graph G with at least two vertices, $\alpha(G) \le 1 + \min_v \mathrm{distEven}(v)\,(\Lambda - 1)$.*

Proof. If $\Lambda = 1$, every neighbourhood is a clique, so the connected graph G is complete and $\alpha = 1$. Assume $\Lambda \ge 2$.

Fix v, put $D = \mathrm{distEven}(v)$, and let L_i be the set of vertices at distance i from v. Assign to each $x \in L_{2j+1}$ a neighbour $\pi(x) \in L_{2j}$. For y at even distance let $C(y) = \pi^{-1}(y) \subseteq N(y) \cap L_{2j+1}$. Then V is the disjoint union of the sets $\{y\} \cup C(y)$.

Let I be independent. If $y \in I$, then $I \cap C(y) = \emptyset$. If $y = v \notin I$, then $|I \cap C(v)| \le l(v) \le \Lambda$. If $y \in L_{2j}$ with $j \ge 1$ and $y \notin I$, choose a neighbour $z \in L_{2j-1}$ of y. This z has no neighbour in L_{2j+1}, so $(I \cap C(y)) \cup \{z\}$ is independent in $G[N(y)]$, and $|I \cap C(y)| \le l(y) - 1 \le \Lambda - 1$.

Summing over the D even-distance vertices gives $|I| \le \Lambda + (D-1)(\Lambda-1) = 1 + D(\Lambda-1)$. □

Priority. A proof by the same BFS-layer idea was posted for review by J. J. Gebendorfer on 28 October 2026 (Formal Conjectures issue #7664). We include our argument only for completeness and claim no priority.

Proposition 2.5 (WOWII 133, C_4 branch)**.** *If G is connected and contains a 4-cycle, then $\mathrm{path}(G) \ge \mathrm{rad}(G) + 1$. Hence WOWII 133 is equivalent to its restriction to C_4-free graphs. In a C_4-free graph $l(v) = d(v) - t(v)$, where $t(v)$ is the number of edges inside $N(v)$.*

Proof. A diametral geodesic is an induced path on $\mathrm{diam} + 1 \ge \mathrm{rad} + 1$ vertices. In a C_4-free graph no vertex of $N(v)$ has two neighbours in $N(v)$, so $G[N(v)]$ is a matching plus isolated vertices. □

Proposition 2.6 (WOWII 189 for diameter ≤ 2)**.** *If $\mathrm{diam}(G) \le 2$ and $\max_v \mathrm{distEven}(v) \le 1 + \sigma(G)$, then G has a Hamiltonian path.*

Proof. For diameter at most two, $\mathrm{distEven}(v) = n - d(v)$, so the hypothesis reads $\delta + \sigma \ge n - 1$. Any two distinct vertices therefore have degree sum at least $n - 1$, and Ore's theorem for Hamiltonian paths applies. □

3 A status note on order-five Austin laws

The Equational Theories Project lists 96 order-five laws whose finite models are trivial and whose infinite models were unknown [5]. Since then, classifications have come from three sources:

- Zhao–Wu–Yang–Miao [6] certified 32 of them as Austin laws, 4 of which overlap the ALPS benchmark positives;
- ALPS [7] has two further positives, 12857/33436;
- corpus item [085] settled 5093/41179 and 5107/40951.

In the same-resource baseline of [6], Twee proved $x \approx y$ from 5834 and from 40037, so this dual pair is trivial rather than Austin. Subtracting all of these leaves 28 dual pairs, i.e. 56 laws, that are unclassified as far as we can determine:

5833/40070	5837/40221	5947/40057	5951/40208
6820/39485	6878/39126	6895/39163	6912/39214
7587/38316	7763/38565	8485/37519	9337/36867
9603/36514	9663/36487	10218/35685	10222/35836
11082/34889	11205/35100	11280/34778	12073/33998
12087/33884	12234/33883	12883/33020	13764/32294
13849/32281	13992/32280	18137/27863	18212/27859

This is bookkeeping, not a theorem. We attempted no classification, because no working equational prover was available in our environment.

4 Assessment

- **Erdős #700(a).** Partial answer. Proposition 1.4 and Theorem 1.5 with Corollaries 1.6–1.7 are new as far as we can determine, and they completely settle the class $\omega(n) \leq 2$. The general question remains open, and the problem's own forum expects no simple answer.
- **WOWII.** Only computational evidence, plus three short arguments. Of these, Theorem 2.4 has an earlier public claim, and the other two are partial. No WOWII conjecture is newly resolved here.
- **Austin laws.** A status list only.

Measured against the corpus criteria (a pre-existing named problem, unchanged and unconditionally and completely resolved), none of the items is a complete resolution. The closest is the complete two-prime characterization for Erdős #700(a).

References

[1] P. Erdős and G. Szekeres, *Some number theoretic problems on binomial coefficients*, Austral. Math. Soc. Gaz. **5** (1978), 97–99.

[2] T. F. Bloom, *Erdős Problem #700*, `https://www.erdosproblems.com/700`, and its discussion thread (accessed 8 November 2026).

[3] The Formal Conjectures Authors, *Formal Conjectures*, `github.com/google-deepmind/formal-conjectures`, files `WrittenOnTheWallII/GraphConjecture{19,40,61,72,96,133,189,199}.lean`.

[4] E. DeLaViña, *Written on the Wall II (Conjectures of Graffiti.pc)*, University of Houston–Downtown.

[5] Equational Theories Project, *Blueprint*, chapter "Order 5 Austin laws", `github.com/teorth/equational_theories`.

[6] J. Zhao, B. Wu, T. Yang and X. Miao, *Trace-tree magmas*, arXiv:2609.05690 (2026); artifacts at `github.com/YanbiaoLab/trace-tree-magmas`.

[7] E. Xie, W. Ye and A. Zhang, *ALPS*, arXiv:2608.15979 (2026).

A Programs

A.1 `f700list.c`: composite $n \leq N$ with $f(n) = n/P(n)$

```
#include <stdio.h>
#include <stdlib.h>
typedef unsigned long long u64;
static int carries(u64 a,u64 b,u64 p,int cap){ int c=0,carry=0; while(a||b||carry){ u64 s=a%p+b%p+carry;
     carry=s>=p; if(carry){c++; if(c>=cap) return cap;} a/=p; b/=p;} return c; }
int main(int argc,char**argv){ u64 N=strtoull(argv[1],0,10);
  for(u64 n=4;n<=N;n++){
    u64 pr[20]; int ex[20],np=0; u64 m=n; for(u64 p=2;p*p<=m;p++) if(m%p==0){pr[np]=p;ex[np]=0;while(m%p
     ==0){m/=p;ex[np]++;}np++;} if(m>1){pr[np]=m;ex[np]=1;np++;}
    if(np==1 && ex[0]==1) continue; /* prime */
```

```
    u64 P=pr[np-1], target=n/P, best=n;
    for(u64 k=2;k<=n/2;k++){ u64 g=1; for(int i=0;i<np;i++){ int c=carries(k,n-k,pr[i],ex[i]); for(int t
     =0;t<c;t++) g*=pr[i]; if(g>=best) break; } if(g<best){best=g; if(best<target) break;} }
    if(best==target && !(np==2 && ex[0]==1 && ex[1]==1)) { printf("%llu =",n); for(int i=0;i<np;i++)
     printf(" %llu^%d",pr[i],ex[i]); printf("\n"); }
  }
}
```

A.2 `thmA.py`: Theorem 1.5 against brute force

```
# Verify Theorem A (Erdos #700(a), omega(n)<=2) against brute force f(n) for all composite n<=N.
import sys
from sympy import factorint
N=int(sys.argv[1])
def carries(a,b,p):
    c=0; carry=0
    while a or b or carry:
        s=a%p+b%p+carry; carry=1 if s>=p else 0; c+=carry; a//=p; b//=p
    return c
def f_brute(n,fac):
    best=n
    for k in range(2,n//2+1):
        g=1
        for p,e in fac.items():
            g*=p**min(e,carries(k,n-k,p))
            if g>=best: break
        if g<best: best=g
    return best
def predict(n,fac):
    ps=sorted(fac)
    if len(ps)==1:
        p,a=ps[0],fac[ps[0]]
        return a==2
    if len(ps)==2:
        p,q=ps  # p<q, q=P(n)
        a,b=fac[p],fac[q]
        if b!=1 or p**a>q: return False
        for t in range(1,a):
            for jp in range(1, p**(a-t)//2*2+1):
                if jp%p==0: continue
                if p**t*jp > p**a//2: break
                if carries(jp*q,(p**(a-t)-jp)*q,p) < a: return False
        return True
    return None
mism=0; cnt=0; true_count=0
for n in range(4,N+1):
    fac=factorint(n)
    if len(fac)==1 and list(fac.values())[0]==1: continue
    if len(fac)>2: continue
    cnt+=1
    P=max(fac)
    actual = (f_brute(n,fac)==n//P)
    pred=predict(n,fac)
    if actual: true_count+=1
    if pred!=actual:
        mism+=1
        if mism<=10: print("MISMATCH",n,fac,actual,pred)
print(f"checked {cnt} composite n<= {N} with omega<=2; f=n/P holds for {true_count}; mismatches={mism}")
```

A.3 `wow.c`: the eight WOWII conjectures on graph6 input

```
/* Exhaustive checker for open WOWII conjectures 19,40,61,72,96,133,189,199
   (definitions follow the Formal Conjectures Lean formalisation). Input: graph6 lines (connected graphs).
    */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef unsigned int U;
```

```
static int n; static U adj[16];
static int pc(U x){ return __builtin_popcount(x); }
static int parse(const char*s){
  n=s[0]-63; memset(adj,0,sizeof adj);
  int k=0; const char*p=s+1; int bits=0,val=0;
  for(int j=1;j<n;j++) for(int i=0;i<j;i++){
    if(bits==0){ val=(*p++)-63; bits=6; }
    bits--; if((val>>bits)&1){ adj[i]|=1u<<j; adj[j]|=1u<<i; }
    k++;
  }
  return n;
}
static U reach(U mask,int s){ /* vertices reachable from s inside mask */
  U seen=1u<<s, fr=seen;
  while(fr){ U nf=0; for(U f=fr;f;f&=f-1){ int v=__builtin_ctz(f); nf|=adj[v]&mask; } nf&=~seen; seen|=nf
     ; fr=nf; }
  return seen;
}
static int connected(U mask){ if(!mask) return 0; return reach(mask,__builtin_ctz(mask))==mask; }
static int edges(U mask){ int e=0; for(U m=mask;m;m&=m-1){ int v=__builtin_ctz(m); e+=pc(adj[v]&mask);}
     return e/2; }
static int ncomp(U mask){ int c=0; while(mask){ U r=reach(mask,__builtin_ctz(mask)); mask&=~r; c++; }
     return c; }
static int bipartite(U mask){ /* 2-colour each component by BFS */
  U rem=mask;
  while(rem){ int s=__builtin_ctz(rem); U A=1u<<s,B=0, frA=A, frB=0, seen=A;
    while(frA||frB){ U nB=0,nA=0;
      for(U f=frA;f;f&=f-1){ int v=__builtin_ctz(f); nB|=adj[v]&mask; }
      for(U f=frB;f;f&=f-1){ int v=__builtin_ctz(f); nA|=adj[v]&mask; }
      if((nB&A)||(nA&B)) return 0;
      frB=nB&~B; frA=nA&~A; B|=nB; A|=nA; seen|=nA|nB; }
    if(A&B) return 0;
    rem&=~seen; }
  return 1;
}
static int indepnum_in(U mask){ /* max independent set inside mask, simple branching */
  if(!mask) return 0;
  int v=__builtin_ctz(mask);
  /* choose vertex of max degree in mask for branching */
  int best=-1,bd=-1; for(U m=mask;m;m&=m-1){ int u=__builtin_ctz(m); int d=pc(adj[u]&mask); if(d>bd){bd=d
     ;best=u;} }
  if(bd==0) return pc(mask);
  v=best;
  int a=indepnum_in(mask&~(1u<<v));            /* exclude v */
  int b=1+indepnum_in(mask&~(1u<<v)&~adj[v]);   /* include v */
  return a>b?a:b;
}
int main(int argc,char**argv){
  char line[256]; long long cnt=0; long long viol[8]={0}; const int ids[8]={19,40,61,72,96,133,189,199};
  int printed[8]={0};
  static unsigned char hp[1<<12][12]; static int pcov[1<<12];
  while(fgets(line,sizeof line,stdin)){
    line[strcspn(line,"\n")]=0; if(!line[0]) continue;
    parse(line); if(n<2) continue; cnt++;
    U full=(n==32)?~0u:((1u<<n)-1);
    /* distances */
    int dist[16][16];
    for(int s=0;s<n;s++){ for(int t=0;t<n;t++) dist[s][t]=-1; dist[s][s]=0; U seen=1u<<s, fr=seen; int d
     =0;
      while(fr){ d++; U nf=0; for(U f=fr;f;f&=f-1) nf|=adj[__builtin_ctz(f)]; nf&=~seen; for(U f=nf;f;f&=
     f-1) dist[s][__builtin_ctz(f)]=d; seen|=nf; fr=nf; } }
    int ecc[16], sumEcc=0, rad=99, diam=0;
    for(int v=0;v<n;v++){ int e=0; for(int t=0;t<n;t++) if(dist[v][t]>e) e=dist[v][t]; ecc[v]=e; sumEcc+=
     e; if(e<rad) rad=e; if(e>diam) diam=e; }
    int lv[16], maxL=0, sumL=0;
    for(int v=0;v<n;v++){ lv[v]=indepnum_in(adj[v]); sumL+=lv[v]; if(lv[v]>maxL) maxL=lv[v]; }
    int alpha=indepnum_in(full);
    int distEven[16], minDE=99, maxDE=0;
    for(int v=0;v<n;v++){ int c=0; for(int t=0;t<n;t++) if(dist[v][t]%2==0) c++; distEven[v]=c; if(c<minDE
     )minDE=c; if(c>maxDE)maxDE=c; }
    /* subset invariants */
    int bG=0,fG=0,treeG=0,pathG=0;
    for(U S=1;S<=full;S++){
```

```
    int k=pc(S);
    int e=edges(S);
    int comps=ncomp(S);
    int acyclic=(e==k-comps);
    if(acyclic){ if(k>fG) fG=k; if(comps==1){ if(k>treeG) treeG=k;
        int maxdeg=0; for(U m=S;m;m&=m-1){ int d=pc(adj[__builtin_ctz(m)]&S); if(d>maxdeg)maxdeg=d; }
        if(maxdeg<=2 && k>pathG) pathG=k; } }
    if(k>bG && bipartite(S)) bG=k;
  }
  /* Hamiltonian path DP */
  for(U S=0;S<=full;S++) for(int v=0;v<n;v++) hp[S][v]=0;
  for(int v=0;v<n;v++) hp[1u<<v][v]=1;
  for(U S=1;S<=full;S++) for(int v=0;v<n;v++) if(hp[S][v]){ U nb=adj[v]&~S; for(U m=nb;m;m&=m-1){ int u
   =__builtin_ctz(m); hp[S|(1u<<u)][u]=1; } }
  int hamS[1<<12]; for(U S=1;S<=full;S++){ int h=0; for(int v=0;v<n;v++) if(hp[S][v]){h=1;break;} hamS[S
   ]=h; }
  int hasHamPath=hamS[full];
  /* path cover number */
  pcov[0]=0;
  for(U S=1;S<=full;S++){ int low=S&(-S); int best=99; U rest=S&~low;
    for(U sub=rest;;sub=(sub-1)&rest){ U T=sub|low; if(hamS[T]){ int c=1+pcov[S&~T]; if(c<best) best=c;
    } if(sub==0) break; }
    pcov[S]=best; }
  int pG=pcov[full];
  /* residue (Havel-Hakimi) */
  int deg[16]; for(int v=0;v<n;v++) deg[v]=pc(adj[v]);
  int seq[16]; for(int i=0;i<n;i++) seq[i]=deg[i];
  int len=n, residue=0;
  while(1){ /* sort descending */
    for(int i=0;i<len;i++) for(int j=i+1;j<len;j++) if(seq[j]>seq[i]){int t=seq[i];seq[i]=seq[j];seq[j
   ]=t;}
    if(len==0){ residue=0; break; }
    if(seq[0]==0){ residue=len; break; }
    int d=seq[0]; for(int i=1;i<=d && i<len;i++) seq[i]--; for(int i=0;i<len-1;i++) seq[i]=seq[i+1];
   len--; }
  /* second smallest degree */
  int sd[16]; for(int i=0;i<n;i++) sd[i]=deg[i]; for(int i=0;i<n;i++) for(int j=i+1;j<n;j++) if(sd[j]<sd
   [i]){int t=sd[i];sd[i]=sd[j];sd[j]=t;}
  int sigma=sd[1];
  /* C4 (not necessarily induced): two distinct vertices with >=2 common neighbours */
  int hasC4=0; for(int a=0;a<n && !hasC4;a++) for(int c=a+1;c<n;c++) if(pc(adj[a]&adj[c])>=2){hasC4=1;
   break;}
  /* vertex connectivity */
  int kappa=n-1;
  for(U S=0;S<full;S++){ int k=pc(S); if(k>=kappa) continue; U R=full&~S; if(!connected(R)) kappa=k; }
  /* --- conjectures --- */
  if(argc>1){ printf("%s n=%d b=%d f=%d tree=%d path=%d p=%d res=%d diam=%d rad=%d sumEcc=%d maxL=%d
   sumL=%d alpha=%d minDE=%d maxDE=%d sigma=%d kappa=%d C4=%d ham=%d\n",line,n,bG,fG,treeG,pathG,pG,
   residue,diam,rad,sumEcc,maxL,sumL,alpha,minDE,maxDE,sigma,kappa,hasC4,hasHamPath); }
  int bad[8]={0};
  /* 19: b >= floor(avgEcc + maxL) */
  { int rhs=(sumEcc + n*maxL)/n; if(bG<rhs) bad[0]=1; }
  /* 40: f >= ceil((p+b+1)/2) */
  { int rhs=(pG+bG+1+1)/2; if(fG<rhs) bad[1]=1; }
  /* 61: f >= residue + ceil(diam/3) */
  { int rhs=residue+(diam+2)/3; if(fG<rhs) bad[2]=1; }
  /* 72: tree >= ceil(avgEcc + maxL/3) = ceil((3 sumEcc + n maxL)/(3n)) */
  { int num=3*sumEcc+n*maxL, den=3*n; int rhs=(num+den-1)/den; if(treeG<rhs) bad[3]=1; }
  /* 96: alpha <= 1 + minDE*(maxL-1) */
  { if(alpha > 1 + minDE*(maxL-1)) bad[4]=1; }
  /* 133: path >= rad + (avgL)~{cC4}; cC4=0 if hasC4 */
  { if(hasC4){ if(pathG < rad+1) bad[5]=1; } else { if(n*pathG < n*rad + sumL) bad[5]=1; } }
  /* 189: maxDE <= 1+sigma => Ham path */
  { if(maxDE<=1+sigma && !hasHamPath) bad[6]=1; }
  /* 199: tree-2 <= kappa => Ham path */
  { if(treeG-2<=kappa && !hasHamPath) bad[7]=1; }
  for(int i=0;i<8;i++) if(bad[i]){ viol[i]++; if(printed[i]<3){ printed[i]++; printf("VIOL %d n=%d %s |
    b=%d f=%d tree=%d path=%d p=%d res=%d diam=%d rad=%d sumEcc=%d maxL=%d sumL=%d alpha=%d minDE=%d
   maxDE=%d sigma=%d kappa=%d C4=%d ham=%d\n",
      ids[i],n,line,bG,fG,treeG,pathG,pG,residue,diam,rad,sumEcc,maxL,sumL,alpha,minDE,maxDE,sigma,
   kappa,hasC4,hasHamPath); fflush(stdout);} }
}
printf("graphs=%lld",cnt); for(int i=0;i<8;i++) printf(" C%d:%lld",ids[i],viol[i]); printf("\n");
```

```
  return 0;
}
```

A.4 12.c: inequality (2)

```
/* Test (L2) f >= res + nu_ind and (L2') alpha2 >= res + nu_ind on connected graphs from stdin */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef unsigned int U; static int n; static U adj[16];
static int pc(U x){return __builtin_popcount(x);}
static void parse(const char*s){ n=s[0]-63; memset(adj,0,sizeof adj); const char*p=s+1; int bits=0,val=0;
  for(int j=1;j<n;j++) for(int i=0;i<j;i++){ if(!bits){val=(*p++)-63;bits=6;} bits--; if((val>>bits)&1){
     adj[i]|=1u<<j;adj[j]|=1u<<i;} } }
static U reach(U mask,int s){U seen=1u<<s,fr=seen; while(fr){U nf=0; for(U f=fr;f;f&=f-1) nf|=adj[
     __builtin_ctz(f)]&mask; nf&=~seen; seen|=nf; fr=nf;} return seen;}
static int ncomp(U m){int c=0; while(m){U r=reach(m,__builtin_ctz(m)); m&=~r; c++;} return c;}
static int eu[64],ev[64],m_; static int best_;
static void rec(int start,U blocked,int sz){ if(sz>best_) best_=sz; for(int t=start;t<m_;t++){ U eb=(1u<<
     eu[t])|(1u<<ev[t]); if(eb&blocked) continue; rec(t+1,blocked|eb|adj[eu[t]]|adj[ev[t]],sz+1);} }
int main(){ char line[256]; long long cnt=0,b2=0,b2p=0; int sh=0;
  while(fgets(line,sizeof line,stdin)){ line[strcspn(line,"\n")]=0; parse(line); if(n<2) continue; cnt++;
    U full=(1u<<n)-1; int f=0,a2=0;
    for(U S=1;S<=full;S++){ int k=pc(S); int e=0,maxd=0; for(U x=S;x;x&=x-1){int d=pc(adj[__builtin_ctz(x
     )]&S); e+=d; if(d>maxd)maxd=d;} e/=2;
      if(maxd<=1 && k>a2) a2=k;
      if(k>f && e==k-ncomp(S)) f=k; }
    m_=0; for(int i=0;i<n;i++) for(int j=i+1;j<n;j++) if(adj[i]>>j&1){eu[m_]=i;ev[m_]=j;m_++;}
    best_=0; rec(0,0,0); int nu=best_;
    int seq[16],len=n,res=0; for(int i=0;i<n;i++) seq[i]=pc(adj[i]);
    while(1){ for(int i=0;i<len;i++) for(int j=i+1;j<len;j++) if(seq[j]>seq[i]){int t=seq[i];seq[i]=seq[j
     ];seq[j]=t;}
      if(len==0){res=0;break;} if(seq[0]==0){res=len;break;} int d=seq[0]; for(int i=1;i<=d&&i<len;i++)
     seq[i]--; for(int i=0;i<len-1;i++) seq[i]=seq[i+1]; len--; }
    if(f<res+nu){ b2++; if(sh++<5){printf("L2 FAIL %s f=%d res=%d nu=%d\n",line,f,res,nu); fflush(stdout)
     ;} }
    if(a2<res+nu){ b2p++; }
  }
  printf("graphs=%lld L2fail=%lld L2'(alpha2)fail=%lld\n",cnt,b2,b2p);
}
```